

Assembler, LED, and Text-Based Virtual Pet

Tangina Kader, Amy Showell, Shriya Sistla, Yao Tong

June 19, 2025

1 Assembler Implementation

Our assembler is composed of three main components: the emulate file, where instruction fetch and encode is performed; the modules, which consist of all instruction-encoding functions; and the data structures. The first data structure is the dynamic **syntable**, a dictionary of labels to addresses. The second data structure is the alias table, a dictionary of mnemonics to function pointers where the instruction type is handled.

This assembler utilizes two-pass assembly. The function **first_pass** loops through each line of the input assembly file, storing labels and their corresponding addresses in a symbol table. Next, the function **second_pass** accesses each line of the assembly file again, tokenising each line to identify the instruction name, replace labels with offsets, and extract instruction parameters. The instruction name is then inputted to the alias table's lookup function, which returns a function pointer to the proper encode function. For example, inputting the **add** instruction to lookup table returns a pointer to function **arith** in module **data-processing**. Each of these functions takes the list of tokens, the size of the token list, and an integer pointer to where the encoded instruction is to be stored. The integer pointer's contents are updated based on the instruction type and tokens. Each function also returns a status code indicating encoding success, either **EXIT_SUCCESS** or **EXIT_FAILURE**. Lastly, the encoded instruction is converted to binary and written to the specified output file. Note that in the second pass, any line consisting of only a label, directive, or newline is automatically skipped.

While writing our assembler, we learned to appreciate the importance of fleshing out useful data structures before implementing other functionality. For example, the dynamic symbol table was originally fixed-length, which is inefficient given there could be any number of labels in an assembly file. Also, the table initially stored addresses as integer pointers instead of just integers, which caused issues when looking up addresses using labels. Thus, thorough brainstorming, testing, and review are always necessary before a data structure is deployed. In addition, our group focused on the challenging implementation task we identified last week: writing utility functions early on. Throughout the assembler task, any function that seemed usable in multiple modules – like extracting immediate values and register numbers – were written in a file called **assembly-utils.c**. As a result, many of our tasks were simplified, and we enjoyed more efficient development.

2 Blinking LED Implementation

To implement Part III, we began by building a simple circuit with one red LED, one 220 Ω resistor, one wire to ground, and one wire to the output at GPIO pin 3. Then, we made a file, **led.blink.s**,

in which we started to write our assembly code. Our assembly code has five main sections: `setout`, `sethigh`, `setlow`, `wait`, and a section for integer directives.

In the first section, `setout`, we select pin 3 as the output by setting FSEL3 to binary 001 and storing this value in the given GPIO function selection address.

The next section, `sethigh`, turns the LED on by storing 0x8 in the GPSET0 register address, which sets the bit corresponding to pin 3. Additionally, we set an “LED state” flag in register w9 to signal that the current state is high before branching to the `wait` section of the code. Similarly, the `setlow` section turns the LED off by storing 0x8 in the GPCLR0 register address, setting the bit corresponding to pin 3 of the CLEAR register. This time, we set the “LED state” flag register to 0 to signal that the current state is low before branching to `wait`.

In the `wait` section of our code, we use a loop to wait for a fixed amount of time before updating the output signal. First, we reset the loop by setting register textttw3 to 0. Within the loop, we add 1 to register w3 until it is greater than or equal to a set wait time of 0x50000. Once the wait time is reached, we check the “LED state” flag register to see if the current state is high or low, branching to the next state depending on the outcome of the comparison.

The final section of our code includes a set of seven integer directives that are loaded into registers at the start of the program. The integer directives include the GPIO function select address, the GPSET0 address, the GPCLR0 address, the wait time, the GPIO configuration for function selection, the GP output configuration for sending high/low signals, and the “LED state” flag.

We faced many challenges while writing our assembly code. For example, our attempts to use the `mov` instruction to load an immediate value into a register yielded incorrect results, which confused us greatly. When we finally realised that our emulator and assembler do not support the functionality for immediate operands in `mov` instructions, we changed the structure of the code to use the `ldr` instruction combined with integer directives to store the set and clear flags in the corresponding GPIO output registers. When using a `mov` instruction, we used register operands rather than immediate values.

Another challenge we faced was understanding the assembly we had written. Since our assembler does not support comments, descriptive variable names and labels were essential to code clarity. However, when we began writing the code, our variable names were extremely generic, causing confusion and many delays in debugging. Therefore, we altered our code to have clear labels for different sections and comprehensible integer directive names, resulting in a more productive debugging process.

3 Text-Based Virtual Pet

A “virtual pet” is a digital toy that often requires care and attention. The most popular example is Bandai Namco’s Tamagotchi, a physical toy with a few interaction buttons and a digital screen where the “pet” is displayed. Similar to a Tamagotchi, our text-based virtual pet extension requires players to feed, clean, and play with their pet to keep their pet happy and alive. However, our implementation also includes an “interaction limit” to prevent users from becoming addicted or dependent on their virtual pets. Hence, our virtual pet extension can be used as a fun way to help children and teens practice responsibility and balance; although users should check on and care for their virtual pet every once in a while, they are not allowed to spend the entire day focusing only on their pet. This concept can then be generalized to any aspect of life, including studies, self-care,

hobbies, and more.

3.1 Virtual Pet Design

The virtual pet has three main categories that the user can interact with: cleanliness, happiness, and hunger. Each category decrements by one bar over a fixed amount of time, but can be improved by the user through keyboard inputs. If any category reaches zero, the current virtual pet cannot continue and the user would need to leave the game or restart with a new virtual pet.

We allowed the user to display their current category scores at any point, so they can check if any categories need particular attention. The program automatically warns the user if any category has only one heart remaining. Furthermore, we designed in-game coins that are regularly rewarded to the user if they keep their pet alive; these coins can be used to purchase gifts for and boost the happiness of their virtual pet. We later added levels as a way of encouraging user interaction; the longer a player plays for, the higher the level they can reach.

Our initial design was a text-based computer game, but we later decided to be more ambitious and create two further versions: one using a GUI and another using a LED HAT on a Raspberry Pi. All three versions shared similar functionality and each new design also had its own additional features. The GUI displays images of the virtual pet which are visible and can change throughout the game. For example, at the start a cartoon cat is displayed and if the user feeds their virtual pet, the image changes to the same cat holding a fish. The Raspberry Pi design is similar but is implemented differently because it involves hardware. This version could be more appealing to users because it uses a smaller screen.

3.2 Challenges

Initially, we ran into some problems implementing the pet as a global structure pointer. Eventually, we realised the main issue was how the files were linked, which led to multiple declarations of the same variable. We fixed it by ensuring the virtual pet would only be created once, at the start of the program. This proved particularly helpful when we implemented in-game coins as a separate global variable, which involved some similar C syntax.

When creating the GUI version of our virtual pet, the primary challenge was setting up the packages and dependencies of our chosen graphical user interface toolkit, GTK. During setup, most online tutorials recommended using `sudo` commands. However, these commands are blocked on the lab computers. As a result, we needed to find a workaround, which involved pulling multiple GitHub repositories that GTK depended on. The next challenge was setting up the makefile to compile C source code that used GTK. This involved much trial and error, and a deeper understanding of makefile flags and macros. In addition, we learned the importance of verifying the existence of vital libraries after setup; when we tried to compile the C source code using version `gtk4`, as suggested by the GTK documentation, the compilation failed. After checking the GTK version, we discovered that the GitHub repository only contained `gtk+-3.0`. If we had checked the version at the very beginning, we could have saved valuable time. Lastly, overcoming the initial GTK learning curve required watching a few YouTube tutorials, but we were able to implement our basic required functionality using just the GTK documentation afterwards.

The Raspberry Pi implementation was difficult at first, since it required a thorough reading of the LED HAT manual, as well as an understanding of the libraries provided with it. The use of these libraries also involved modifying our Makefile to link them, and this was a challenging task.

To overcome this, we examined the Makefile provided in the demonstration code for the LED HAT, understanding the purpose of each line to craft our own program's Makefile. Furthermore, after writing code to take user input from the buttons on the LED HAT, a button press triggered too many calls to the user actions functions, causing the statistics of the pet to maximise with only a single press of the button. To solve this problem, we had to find the perfect wait time between recognising a user input and calling a user actions function so that the statistics were only modified once.

3.3 Testing

When we started designing our extension, we listed desirable functionalities we hoped to achieve. These included the user interacting with the virtual pet through the keyboard inputs, decreasing the various category statistics (such as happiness) at regular time intervals, and limiting the number of hearts in each category between zero and five. Later, we referred back to this checklist to thoroughly test the program.

First, we confirmed that the stats would decrease at the expected regular intervals: every 12 seconds for cleanliness, 8 seconds for happiness, and 5 seconds for hunger. To do so, we started the game without interacting with the category being tested. We printed the stats at the expected time intervals to test if the stat for the current category was decrementing each time, and observed the final outcome of the virtual pet dying when the category reached zero hearts.

To test the user inputs, we waited for a particular category to have fewer than five hearts, then input the appropriate character. This test succeeds if we obtain an output message stating this category had increased as expected. For example, typing 'f' or 'F' should print an output message incrementing the hunger score. Similar tests were carried out across the GUI and Raspberry Pi versions, as well as additional implementation-specific checks such as testing that the image displayed on the GUI changed as expected for particular keyboard inputs.

We performed many more manual tests to determine whether the program worked as expected. In the future, we hope to test more rigorously by creating a test suite, as was used for the emulator and assembler, to test many possible states of our program and ensure it always operates in a predictable and desirable way. Due to time restrictions in this project, we decided it would be better to test the program in some of the most likely scenarios, and confirm if the main functionality works rather than risk running out of time building a test suite and being unable to test anything at all.

4 Group Reflection

5 Individual Reflections

5.1 Tangina Kader

5.2 Amy Showell

I tried to communicate often with the others both online and in real life, which helped with group discussions and moving the project forward effectively. Through this project, I've realised that I have a tendency to rush and overlook details which can lead to code that is difficult to read or doesn't pass all the tests. Thankfully Shriya and Yao pay more attention to detail than me,

and notice minor mistakes in my code which I missed; these could have easily become larger problems if they hadn't been caught until much later. In future, I will slow down and focus more on the quality of my work as well as the quantity. Overall, I think I fit in well with this group. I engaged in group/pair programming and debugging several times, particularly for Part III. I clearly communicated which tasks I was working on and attempted to understand what everyone else was doing by asking questions and using other resources, such as Scientia.

5.3 Shriya Sistla

5.4 Yao Tong